

NAME : DISHA SAMANTA

CUID : CU24250227

ROLLNO : 20

LABSHEET 9

```
!pip install ultralytics opencv-python matplotlib
```

```
Collecting ultralytics
  Downloading ultralytics-8.4.154-py3-none-any.whl.metadata (46 kB)
    46.5/46.5 kB 2.1 MB/s eta 0:00:00
Requirement already satisfied: opencv-python in /usr/local/lib/python3.13/dist-packages (5.0.0.93)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages (3.10.0)
Requirement already satisfied: cloudpickle>=3.1.1 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (3.1.2)
Requirement already satisfied: filelock>=3.16.1 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (3.32.5)
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.1.3)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (11.3.0)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (6.0.3)
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.32.4)
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.11.0+cpu)
Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (0.26.0+cpu)
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (5.9.5)
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (1.35.2)
Collecting nvidia-ml-py>=12.0.0 (from ultralytics)
  Downloading nvidia_ml_py-13.610.43-py3-none-any.whl.metadata (9.7 kB)
Collecting ultralytics-thop>=2.1.6 (from ultralytics)
  Downloading ultralytics_thop-2.1.6-py3-none-any.whl.metadata (13 kB)
Collecting ultralytics-platform>=0.1.45 (from ultralytics)
  Downloading ultralytics_platform-0.1.46-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (4.64.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (26.3)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.13/dist-packages (from polars>=0.20.0->ultralytics) (1.35.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (3.19)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (2026.7.7)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (4.16.0)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (80.10.2)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (2025.12.0)
Requirement already satisfied: httpx<1,>=0.28 in /usr/local/lib/python3.13/dist-packages (from ultralytics-platform>=0.1.45->ultralytics) (0.28.0)
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.28->ultralytics-platform>=0.1.45->ultralytics) (4.8.0)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.28->ultralytics-platform>=0.1.45->ultralytics) (1.0.7)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.*->httpx<1,>=0.28->ultralytics-platform>=0.1.45->ultralytics) (0.14.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.13/dist-packages (from sympy>=1.13.3->torch>=1.8.0->ultralytics) (3.1.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from Jinja2->torch>=1.8.0->ultralytics) (3.0.3)
Downloading ultralytics-8.4.154-py3-none-any.whl (1.4 MB)
    1.4/1.4 MB 11.8 MB/s eta 0:00:00
Downloading nvidia_ml_py-13.610.43-py3-none-any.whl (53 kB)
    53.2/53.2 kB 4.1 MB/s eta 0:00:00
Downloading ultralytics_platform-0.1.46-py3-none-any.whl (78 kB)
    78.1/78.1 kB 3.3 MB/s eta 0:00:00
Downloading ultralytics_thop-2.1.6-py3-none-any.whl (30 kB)
Installing collected packages: nvidia-ml-py, ultralytics-thop, ultralytics-platform, ultralytics
Successfully installed nvidia-ml-py-13.610.43 ultralytics-8.4.154 ultralytics-platform-0.1.46 ultralytics-thop-2.1.6
```

```
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO
```

Creating new Ultralytics Settings v0.0.8 file 
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see <https://docs.ultralytics.com/usage>

```
model = YOLO("yolov8n.pt")
```

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt> to 'yolov8n.pt': 100% ————— 6.2MB 47.9MB/s 0.

```
!wget -q https://ultralytics.com/images/bus.jpg
```

```
img = cv2.imread("bus.jpg")
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
plt.figure(figsize=(10,6))
plt.imshow(img_rgb)
plt.axis("off")
plt.show()
```



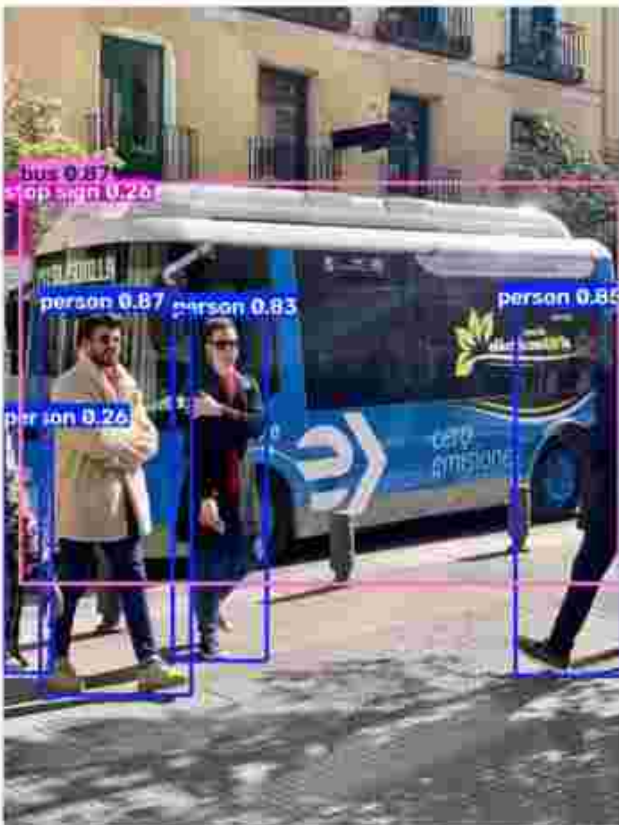
```
results = model("bus.jpg")
```

```
image 1/1 /content/bus.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 428.7ms
Speed: 28.4ms preprocess, 428.7ms inference, 38.7ms postprocess per image at shape (1, 3, 640, 480)
```

```
result = results[0]

annotated_img = result.plot()

plt.figure(figsize=(12,8))
plt.imshow(cv2.cvtColor(annotated_img, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```



```
for box in result.boxes:
    class_id = int(box.cls[0])
    confidence = float(box.conf[0])
```

<pre>class_name = model.names[class_id] print(f"Object: {class_name}, Confidence: {confidence:.2f}")</pre>
<pre>Object: bus, Confidence: 0.87 Object: person, Confidence: 0.87 Object: person, Confidence: 0.85 Object: person, Confidence: 0.83 Object: person, Confidence: 0.26 Object: stop sign, Confidence: 0.26</pre>
<pre>print("Total objects detected:", len(result.bboxes))</pre>
<pre>Total objects detected: 6</pre>
<pre>for box in result.bboxes: print("Bounding Box:", box.xyxy[0].tolist())</pre>
<pre>Bounding Box: [22.87126922607422, 231.27731323242188, 805.0025634765625, 756.8404541015625] Bounding Box: [48.55046463012695, 398.5522155761719, 245.34556579589844, 902.7026977539062] Bounding Box: [669.472900390625, 392.1859436035156, 809.7201538085938, 877.0354614257812] Bounding Box: [221.51730346679688, 405.7985534667969, 344.97064208984375, 857.53662109375] Bounding Box: [0.0, 550.5250244140625, 63.0069580078125, 873.4429321289062] Bounding Box: [0.05817389488220215, 254.45938110351562, 32.557411193847656, 324.8741760253906]</pre>
<pre>import time start = time.time() results = model("bus.jpg") end = time.time() print("Inference Time:", round(end - start, 4), "seconds")</pre>
<pre>image 1/1 /content/bus.jpg: 640x480 4 persons, 1 bus, 1 stop sign, 162.3ms Speed: 7.0ms preprocess, 162.3ms inference, 1.2ms postprocess per image at shape (1, 3, 640, 480) Inference Time: 0.1891 seconds</pre>

Questions and Answers

Q1. Differentiate between image classification, object detection, and image segmentation.

Answer: Image classification identifies the main category or class of an image and gives a class label as output. Object detection identifies multiple objects in an image and localizes them using bounding boxes along with class labels and confidence scores. Image segmentation classifies individual pixels of an image and produces pixel-level masks for different objects.

Q2. Explain the working principle of the YOLO (You Only Look Once) object detection algorithm.

Answer: YOLO (You Only Look Once) is a single-stage object detection algorithm. It processes the complete image through a neural network in a single pass and predicts the bounding box coordinates, object classes, and confidence scores. The predicted bounding boxes are then filtered using Non-Maximum Suppression (NMS) to remove duplicate detections. Since YOLO processes the image in a single pass, it provides fast and efficient object detection and is suitable for real-time applications.

Q3. Compare YOLO, SSD, and Faster R-CNN based on speed, accuracy, and practical applications.

Answer: YOLO and SSD are single-stage object detection algorithms and are generally faster than Faster R-CNN. YOLO provides high speed and good accuracy, making it suitable for real-time applications such as surveillance, robotics, and autonomous systems. SSD also provides fast detection and can be used in real-time applications. Faster R-CNN is a two-stage detector that generally provides high detection accuracy but requires more computational resources and is slower than YOLO and SSD. It is mainly suitable for applications where detection accuracy is more important than real-time speed.

Q4. What is a bounding box? Why is it important in object detection?

Answer: A bounding box is a rectangular box drawn around a detected object in an image. It represents the location and size of the object using coordinates such as the top-left and bottom-right points. Bounding boxes are important because they allow the object detection system to identify not only what the object is but also where it is located in the image.

Q5. Explain the significance of confidence score and Intersection over Union (IoU) in object detection.

Answer: A confidence score represents how confident the detection model is that a particular object belongs to a predicted class. A higher confidence score generally indicates a more reliable detection.

Intersection over Union (IoU) measures the overlap between the predicted bounding box and the actual ground-truth bounding box. It is calculated as:

$$IoU = \text{Area of Intersection} / \text{Area of Union}$$

A higher IoU value indicates that the predicted bounding box is closer to the actual object location.

Q6. What are the advantages of using pre-trained deep learning models over traditional image processing techniques?

Answer: Pre-trained deep learning models can automatically learn complex features from images and usually provide better performance on difficult object detection tasks. They require less training time because they have already learned useful visual features from large datasets. They can also be fine-tuned for specific applications and reduce the need for designing manual features. Compared with traditional image processing techniques, they are generally more robust to changes in object size, position, orientation, and background.

Q7. Why are datasets such as MS COCO and Pascal VOC widely used for object detection research?

Answer: MS COCO and Pascal VOC are widely used because they contain large collections of annotated images with multiple object categories. They provide bounding box annotations and standardized evaluation methods. These datasets allow researchers and developers to train, test, and compare different object detection models using common benchmarks.

Q8. Mention five real-world applications of object detection in computer vision.

Answer: Five real-world applications of object detection are:

1. Autonomous vehicles for detecting cars, pedestrians, and traffic signs.
2. Surveillance systems for detecting and monitoring people and vehicles.
3. Healthcare applications for detecting abnormalities in medical images.
4. Retail analytics for detecting products and monitoring shelves.
5. Industrial inspection for detecting defects in manufactured products.

Q9. What challenges are commonly encountered while detecting objects in complex real-world environments?

Answer: Object detection systems may face several challenges in real-world environments. These include poor lighting, occlusion, complex backgrounds, small objects, crowded scenes, motion blur, different object orientations, variations in object size, and objects with similar appearances. Limited computing resources can also affect the speed and performance of detection systems.

Q10. How can object detection systems be further improved using recent advancements in deep learning and computer vision?

Answer: Object detection systems can be improved by using larger and better-quality datasets, data augmentation, transfer learning, improved model architectures, attention mechanisms, multi-scale detection, and better feature extraction techniques. Model pruning and quantization can reduce computational requirements. Recent advancements such as transformer-based vision models and improved object detection architectures can also help improve detection accuracy, speed, and robustness.